TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÀI THỰC HÀNH 1

CS106

GVHD : LƯƠNG NGỌC HOÀNG

SINH VIÊN THỰC HIỆN : PHAN VŨ ANH TUẤN-23521727

1) Mô hình hóa trò chơi Sokoban:

a. *Trạng thái khởi đầu:*

Trạng thái khởi đầu mô tả vị trí ban đầu của người chơi và các hộp trong bản đồ. Dưới đây là các hàm xác định trạng thái khởi đầu:

```
def PosOfPlayer(gameState):
    """Return the position of agent"""
    return tuple(np.argwhere(gameState == 2)[0]) # e.g. (2, 2)

def PosOfBoxes(gameState):
    """Return the positions of boxes"""
    return tuple(tuple(x) for x in np.argwhere((gameState == 3) | (gameState == 5)))
```

Trong đó, • PosOfPlayer: Xác định vị trí của người chơi • PosOfBoxes: Xác định tất cả các vị trí của các hộp (3 cho hộp thường, 5 cho hộp đã nằm trên mục tiêu). Sau đó khởi tạo trạng thái ban đầu:

```
beginBox = PosOfBoxes(gameState)
beginPlayer = PosOfPlayer(gameState)

startState = (beginPlayer, beginBox)
```

Trong đó:

startState: Là một tuple gồm:

• Vị trí người chơi: beginPlayer

• Danh sách vị trí của các hộp: beginBox

b. Trạng thái kết thúc:

Trạng thái kết thúc là khi các hộp nằm trên các ô mục tiêu

```
def PosOfGoals(gameState):
    """Return the positions of goals"""
    return tuple(tuple(x) for x in np.argwhere((gameState == 4) | (gameState == 5))) # e.g. like those above

def isEndState(posBox):
    """Check if all boxes are on the goals (i.e. pass the game)"""
    return sorted(posBox) == sorted(posGoals)
```

• PosOfGoals: Trích ra tất cả vị trí của mục tiêu

• isEndState: So sánh vị trí của các hộp posBox với posGoals. Nếu tất cả hộp đã nằm trên mục tiêu → kết thúc.

c. Không gian trạng thái:

Bao gồm : vị trí người chơi, vị trí các họp, vị trí tường, vị trí các ô mục tiêu.

d. Các hành động hợp lệ:

Có thể đi lên xuống trái phải, chữ cái in hoa cho biết có đẩy thùng không

```
def legalActions(posPlayer, posBox):
    """Return all legal actions for the agent in the current game state"""
    allActions = [[-1,0,'u','U'],[1,0,'d','D'],[0,-1,'l','L'],[0,1,'r','R']]
    xPlayer, yPlayer = posPlayer
    legalActions = []
    for action in allActions:
        x1, y1 = xPlayer + action[0], yPlayer + action[1]
        if (x1, y1) in posBox: # the move was a push
            action.pop(2) # drop the little letter
        else:
            action.pop(3) # drop the upper letter
        if isLegalAction(action, posPlayer, posBox):
            legalActions.append(action)
        else:
            continue

    return tuple(tuple(x) for x in legalActions) # e.g. ((0, -1, 'l'), (0, 1, 'R'))
```

e. *Hàm tiến triển:*

Hàm tiến triển trong bài toán Sokoban chính là hàm updateState, hàm này chịu trách nhiệm cập nhật trạng thái của trò chơi sau mỗi lần thực hiện một hành động.

```
def updateState(posPlayer, posBox, action):
    """Return updated game state after an action is taken"""
    xPlayer, yPlayer = posPlayer # the previous position of player
    newPosPlayer = [xPlayer + action[0], yPlayer + action[1]] # the current position of player
    posBox = [list(x) for x in posBox]
    if action[-1].isupper(): # if pushing, update the position of box
        posBox.remove(newPosPlayer)
        posBox.append([xPlayer + 2 * action[0], yPlayer + 2 * action[1]])
    posBox = tuple(tuple(x) for x in posBox)
    newPosPlayer = tuple(newPosPlayer)
    return newPosPlayer, posBox
```

Hàm trả về vị trí mới của người chơi và thùng dựa theo action đã thực hiện.

2. Thống kê các bước thực hiện

	DFS	BFS	UCS

	DFS		BFS		UCS	
Level	Time	Steps	Time	Steps	Time	Steps
1	0.06805	79	0.08141	12	0.13197	12
2	0.00404	24	0.00851	9	0.02026	9
3	0.1	403	0.17095	15	0.18665	15
4	0.00208	27	0.00823	7	0.01286	7
5	Treo		167.0893	20	145.3135	20
6	0.01256	55	0.01578	19	0.03814	19
7	0.48704	707	0.76433	21	1.09493	21
8	0.0702	323	0.19601	97	0.45544	97
9	0.23138	74	0.02388	8	0.01569	8
10	0.01401	37	0.02451	33	0.04689	33
11	0.01327	36	0.04794	34	0.04865	34
12	0.13035	109	0.08697	23	0.18274	23
13	0.1863	185	0.1423	31	0.38788	31
14	3.6911	865	2.26924	23	5.9656	23
15	0.15211	291	0.23818	105	0.60553	105
16	Treo		18.61399	34	19.7216	34
17	22.85647	Rỗng	19.43165	không có	19.2784	Không có
18	Treo		Treo		Treo	

3. Nhận xét:

DFS: đây là thuật toán đi sâu vào 1 hướng vì nó sẽ đi theo hướng đã được định sẵn ban đầu mà nếu cái nhánh đó sai thì bfs phải thử từng trạng thái xong mới quay trở về, hoặc nếu có đúng đi nữa thì bfs đi rất vòng vo khiến thời gian tìm ra lời giải rất lâu ví dụ lv3 tốn tới 403 bước trong khi ucs và bfs chỉ mất 15 bước.

BFS: thuật toán giúp tìm ra đường đi tối ưu số bước nhỏ hơn dfs rất nhiều nhưng nhược điểm chí mạng là nếu cây trạng thái khổng lồ dfs sẽ lan và mở lá liên tục dẫn đến treo, ví dụ ở lv18 có tới 10 cái thùng, có vô vàn trạng thái nên bfs treo

UCS: mặc dù UCS cho ra kết quả giống bfs nhưng vì do bài này chi phí đi là như nhau nên ucs giống bfs và vượt trội hơn dfs.

4. Thuật toán tốt nhất:

Là thuật toán bfs vì các bước đi chi phí như nhau mà không tốn thời gian tính chi phí để ưu tiên như ucs và nó tìm ra lời giải tối ưu

5. Bản đồ khó nhất :

Theo em là bản đồ 18 do các được nó hẹp và nó có tới 10 cái thùng, với vô vàn vị trí mà thùng có thể nằm các thùng nằm ở từng vị trí đã là 1 trạng thái chỉ cần điều chỉnh nhỏ đã tạo ra 1 trạng thái mới làm cho ucs và bfs tự tạo ra phân thân của các trạng thái dẫn đến quá tải, treo. Còn với dfs, do có rất nhiều trạng thái mà dfs nó đi sâu vô 1 nhánh và nhánh đó rất sâu tới lúc biết sai thì quay lại lúc trước đó trạng thái xong lại phải đi sâu vô nhánh còn lại. Mà với 10 cái thùng thì đi theo kiểu vậy sẽ sai liên tục đi tới đi lui như 1 người cầu may sẽ tìm được lối ra trong 1 mê cung khổng lồ.